

Introduction to SQL

What is Database ?

- A database is an organized collection of structured information, or data, typically stored electronically in a computer system.
- A database is usually controlled by a database management system (DBMS).
- Data within the most common types of databases in operation today is typically modeled in rows and columns in a series of tables to make processing and data querying efficient.
- SQL is a programming language used by nearly all relational databases to query, manipulate, and define data, and to provide access control.

What is SQL

- SQL stands for Structured Query Language
- SQL lets you access and manipulate databases
- SQL became a standard of the American National Standards Institute (ANSI) in 1986, and of the International Organization for Standardization (ISO) in 1987
- SQL is a popular relational database language first standardized in 1986 by the American National Standards Institute (ANSI).
- Since then, it has been formally adopted as an International Standard by the International Organization for Standardization (ISO) and the International Electro-technical Commission (IEC).

What is SQL

- In particular, it defines the structures and operations of a data model implemented by the DBMS, as well as other components that support data definition, data access, security, programming language interface, and data administration.
- The SQL standard specifies data definition, data manipulation, and other associated facilities of a DBMS that supports the relational data model.

History

- IBM Sequel language developed as part of System R project at the IBM San Jose Research Laboratory
- Renamed Structured Query Language (SQL)
- ANSI and ISO standard SQL:
 - SQL-86
 - SQL-89
 - SQL-92
 - SQL:1999 (language name became Y2K compliant!)
 - SQL:2003
- Commercial systems offer most, if not all, SQL-92 features, plus varying feature sets from later standards and special proprietary features.
 - Not all examples here may work on your particular system.

What Can SQL do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

SQL Parts

- DML -- provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- integrity – the DDL includes commands for specifying integrity constraints.
- View definition -- The DDL includes commands for defining views.
- Transaction control –includes commands for specifying the beginning and ending of transactions.
- Embedded SQL and dynamic SQL -- define how SQL statements can be embedded within general-purpose programming languages.
- Authorization – includes commands for specifying access rights to relations and views.

Types of Query Language

- **DQL:** The data query language (DQL) is the most familiar and is used to run queries on databases and extract information from stored data. For example, selecting and returning the maximum value in a column.
- **DDL:** A data definition language (DDL) is used to codify a database's particular structures and schemas. Creating a table or defining data types is an example.
- **DCL:** A data control language (DCL) defines access, authorizations, and permissions for users and processes accessing the database, including granting administrator privileges, or restricting users to read-only privileges only.
- **DML:** And finally, a data manipulation language (DML) is used to make modifications on existing components of a database, like inserting records, updating values in cells, or deleting data.

DDL (Data Definition Language)

- The SQL data-definition language (DDL) allows the specification of information about relations, including:
 - The schema for each relation.
 - The type of values associated with each attribute.
 - The Integrity constraints
 - The set of indices to be maintained for each relation.
 - Security and authorization information for each relation.
 - The physical storage structure of each relation on disk.

Integrity Constraints in Create Table

- The SQL data-definition language (DDL) allows the specification of information about relations, including:
- The SQL data-definition language (DDL) allows the specification of information about relations, including:
 - Not null
 - Primary Key
 - Foreign Key

Domain Types in SQL

- **char(*n*)**. Fixed length character string, with user-specified length *n*.
- **varchar(*n*)**. Variable length character strings, with user-specified maximum length *n*.
- **nvarchar(*n*)**. The key difference between varchar and nvarchar is the way they are stored, varchar is stored as regular 8-bit data(1 byte per character) and nvarchar stores data at 2 bytes per character. Due to this reason, nvarchar can hold upto 4000 characters and it takes double the space as SQL varchar.
- **int**. Integer (a finite subset of the integers that is machine-dependent).
- **TINYINT(*SIZE*)**. Small integer (a machine-dependent subset of the integer domain type).
- **DOUBLE(*p,d*)**. Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point. (ex., **numeric(3,1)**, allows 44.5 to be stores exactly, but not 444.5 or 0.32)

Domain Types in SQL

- **real, double precision(SIZE,d).** Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(n).** Floating point number, with user-specified precision of at least n digits.
- **Decimal(Size,d).** An exact fixed-point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the d parameter. The maximum number for *size* is 65. The maximum number for d is 30. The default value for *size* is 10. The default value for d is 0.

Domain Types in SQL

- **date:** A calendar date containing a (four-digit) year, month, and day of the month.
- **time:** The time of day, in hours, minutes, and seconds. A variant, `time(p)`, can be used to specify the number of fractional digits for seconds (the default being 0). It is also possible to store time-zone information along with the time by specifying time with `timezone`.
- **DATETIME:** A date and time combination. Format: `YYYY-MM-DD hh:mm:ss`. The supported range is from `'1000-01-01 00:00:00'` to `'9999-12-31 23:59:59'`. Adding `DEFAULT` and `ON UPDATE` in the column definition to get automatic initialization and updating to the current date and time
- **Timestamp (DateTime):** A combination of date and time. A variant, `timestamp(p)`, can be used to specify the number of fractional digits for seconds (the default here being 6). Time-zone information is also stored if with `timezone` is specified.

Date and time values can be specified like this:

`date = '2018-04-25'`

`time = '09:30:00'`

`timestamp (DateTime) = '2018-04-25 10:29:01.45'`

Create Database in SQL

- The CREATE DATABASE statement is used to create a new SQL database.

- **Syntax:**

CREATE DATABASE *databasename*;

- Example

CREATE DATABASE [MY_First_DB]

Delete Database

- The DROP DATABASE statement is used to drop an existing SQL database.

- **Syntax:**

DROP DATABASE *databasename*;

- Example

DROP DATABASE [MY_First_DB]

Create Table Construct

- An SQL relation is defined using the **create table** command:

create table r

$(A_1 D_1, A_2 D_2, \dots, A_n D_n,$
 $(\text{integrity-constraint}_1),$
 $\dots,$
 $(\text{integrity-constraint}_k))$

- r is the name of the relation
- each A_i is an attribute name in the schema of relation r
- D_i is the data type of values in the domain of attribute A_i

Example:

```
create table instructor (  
ID char(5),  
name varchar(20),  
dept_name varchar(20),  
salary numeric(8,2));
```

With Primary and foreign key

```
Create Table Persons  
( PersonID int NOT NULL,  
LastName varchar(255) NOT NULL,  
FirstName varchar(255) NOT NULL,  
City varchar(255),  
Address1 varchar(255),  
PRIMARY KEY (PersonID)  
FOREIGN KEY (City) REFERENCES  
Cities(City)  
);
```

Updates to tables

- **Drop Table**

- **drop table** [Table_Name]

- **Alter**

- **alter table** [Table_Name] **add** A D

- where A is the name of the attribute to be added to relation r and D is the domain of A .
 - All exiting tuples in the relation are assigned *null* as the value for the new attribute.

- **alter table** [Table_Name] **drop** A

- where A is the name of an attribute of relation r
 - Dropping of attributes not supported by many databases

Examples

- Example:

Alter table *instructor*
Add Joining_date datetime

Add a new column of Joining Date in Table Instructor

- Example:

Alter table *instructor*
Drop column name

Dropping column **Name** from table instructor.

Examples

- Updating the column data type

Alter table *instructor*

MODIFY COLUMN name TEXT(50)

Changing Data type of column **name** from **varchar(50)** to **TEXT(50)**

Scenario -1

- Create Database with your enrollment
- Drop Database
- Create a Table of person
- Add a new column DateOfBirth
- Add a new Column of Age
- Add a new Column of state
- Change the data type of Address varchar(512)
- Remove column State from the table